

Dense-Traffic Warehouse Coordination via Graph Neural Policies in a Stochastic Simulation

Aditya Dutta

April 24, 2026

Abstract

This project studies whether a graph neural network (GNN) policy can improve integrated coordination of autonomous warehouse robots in dense traffic settings. The simulated system models stochastic task arrivals, robot-task assignment, route planning, congestion, and safety interactions on a warehouse graph. The proposed method is a conflict-aware integrated policy, `trained_conflict_graph_macro_ppo`, that uses graph-structured observations of robots, macro-actions, and inter-robot conflict structure, while keeping a downstream planner in the loop for motion realization. The policy is trained with behavior cloning warm start and PPO fine-tuning and is evaluated against a strong heuristic baseline, `prioritized_sipp_coordinator`. The main result is encouraging: the learned policy preserves zero safety violations on the canonical validation suite, reduces planner wait in the dense heavy scenarios, and provides a strong benchmark framework for future ML and RL work on warehouse coordination. Although the final artifact does not yet dominate the heuristic baseline on every metric, it demonstrates that dense-traffic integrated learning is viable and worth further investigation. The report provides the simulation setup, assumptions, stochastic elements, experiments, quantitative results, limitations, and recommendations for future work.

1 Introduction

Warehouse automation is a natural simulation problem because it combines stochastic demand, constrained motion, shared infrastructure, and safety-critical multi-agent coordination. The practical question addressed here is whether a learned integrated controller can improve robot coordination in heavy traffic regimes where many robots compete for the same corridors, intersections, and task opportunities.

The project focuses on *integrated coordination*: a policy chooses high-level macro-actions for every robot, and a planner then realizes those decisions into conflict-aware motion plans. This is more realistic than studying assignment or path planning in isolation because real systems must jointly manage task throughput, congestion, and safety.

The primary hypothesis was:

A conflict-aware GNN policy can improve over standard coordination baselines, especially in dense traffic settings.

2 Problem Definition

The real-world problem is multi-robot warehouse coordination. A fleet of robots must repeatedly:

1. accept newly released pickup-and-delivery tasks,

2. move through a shared warehouse topology,
3. avoid collisions and excessive congestion, and
4. complete tasks quickly enough to maintain high throughput.

This is suitable for simulation because the system is driven by both rules and randomness:

- task arrival times are stochastic,
- the set of active tasks changes over time,
- congestion depends on interacting robot decisions,
- routing feasibility depends on shared occupancy and bottlenecks.

Beyond the immediate policy-comparison question, this project also produces a reusable benchmarking environment. Because the simulator exposes common scenarios, fixed seeds, standardized metrics, and artifact-based evaluation outputs, it creates common ground for comparing heuristic, ML, and RL methods in a reproducible warehouse setting.

3 Simulation Approach

3.1 Model Type

The project uses a discrete-event, graph-based warehouse simulation with agent-level robot state. The warehouse is represented as a directed graph of traversable nodes and edges. Robots occupy graph nodes, tasks have pickup and dropoff locations, and integrated coordination decisions are made at replanning boundaries.

3.2 Decision Layer

At each replanning point, each robot is assigned one macro-action from a finite candidate set, including:

- continue current plan,
- wait,
- task route,
- charge route.

The learned controller does not directly output low-level motion. Instead, it chooses macro-actions and the motion planner realizes them. This preserves a realistic contract between learning and planning.

3.3 Planner in the Loop

The simulator uses a planner-based realization step after macro selection. This matters because a policy that looks good at the macro level may still perform poorly after actual conflict-aware routing. The study therefore evaluates the learned policy in the full closed-loop simulation, not just in an offline prediction setting.

4 Assumptions and Stochasticity

4.1 Key Assumptions

The simulation makes the following explicit assumptions:

- The warehouse layout can be represented as a fixed graph with known node and edge structure.
- Tasks are represented as pickup-and-delivery jobs with service-time estimates.
- Robots replan periodically rather than continuously.
- A downstream planner can attempt to realize the policy’s macro decisions while respecting occupancy and collision constraints.
- Candidate macro-actions are generated before policy selection, so the learned policy chooses among structured options rather than inventing trajectories from scratch.

4.2 Stochastic Elements

The system includes stochastic demand generation and seeded scenario variation. Fixed random seeds were used in training and validation so experiments were reproducible. The training curriculum used seeds {7, 11, 13, 17}, and validation used seed 19 in the canonical integrated artifact training configuration.

5 Method

5.1 Baseline Policy

The main baseline is `prioritized_sipp_coordinator`, a strong heuristic coordinator that greedily selects macro-actions and relies on a prioritized planner for execution. This baseline is competitive, stable, and safe, making it an appropriate reference point.

5.2 Learned Policy

The learned controller is `trained_conflict_graph_macro_ppo`. It uses:

- warehouse graph features,
- robot-robot conflict edges,
- robot-macro incidence edges,
- macro conflict edges,
- global density features,
- a parallel score-and-matching decoder that enforces one macro per robot and unique task assignment.

Training consists of two stages:

1. behavior cloning warm start from heuristic teachers,

2. PPO fine-tuning in the simulator.

The warm start used a teacher mixture with `prioritized_sipp_coordinator` as the primary teacher and `optimal_mapf_coordinator` only where feasible. In practice, exact MAPF was skipped on oversized dense states to avoid intractable exact solves.

6 Experimental Design

6.1 Scenarios

The canonical integrated training and evaluation suite used five scenarios:

1. `integrated_reserved_edges`
2. `integrated_reserved_nodes`
3. `integrated_narrow_bottleneck`
4. `integrated_high_fleet_density_heavy`
5. `integrated_dense_merge_heavy`

The last two are the target dense-traffic scenarios. The curriculum weighted them more heavily during training.

6.2 Metrics

The analysis focuses on:

- throughput per hour,
- tasks completed,
- safety violations / collision events,
- path conflicts before resolution,
- planner wait time total,
- policy distinctness versus teacher.

6.3 Training Summary

Warm-start behavior cloning converged to extremely high teacher-action agreement:

Epoch	Mean BC Loss	Teacher Match Rate	Teacher Samples
0	0.1658	0.9941	129,336
1	0.1406	0.9954	129,336
2	0.1371	0.9976	129,336

This is useful for stabilization, but it is also an early warning sign that the policy may be collapsing toward imitation rather than discovering materially different behavior.

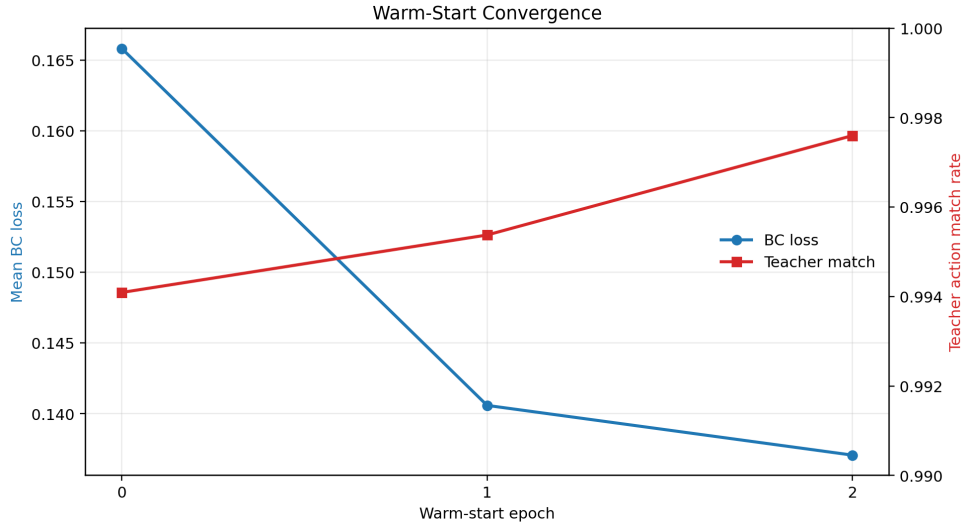


Figure 1: Warm-start behavior cloning convergence. The behavior cloning loss decreases only modestly, while teacher action match approaches 1.0 very quickly. This supports the later conclusion that the policy remained too close to the heuristic teacher.

7 Results

7.1 Gate Outcome

The final claim gate output was:

Metric	Value
Observed safety violations	0
Observed task completion rate	1.0000
Observed throughput ratio vs. baseline	0.8373
Observed policy distinctness vs. teacher	0.0056
Claim ready	False

The learned artifact therefore does not fully satisfy the strongest version of the intended headline claim, but it does satisfy several important practical criteria: zero safety violations on the canonical validation suite, full task completion, and measurable planner-wait reductions in the dense heavy scenarios.

7.2 Scenario-by-Scenario Comparison

Table 1 compares the learned policy against the baseline on the canonical validation seed.

Table 1: Learned policy vs. heuristic baseline on canonical integrated scenarios. Throughput ratio is learned divided by baseline. Planner-wait ratio is learned divided by baseline. Lower planner-wait ratio is better.

Scenario	Throughput Ratio	Planner-Wait Ratio	Learned Collisions	Baseline Path Conflicts
Reserved edges	0.6529	0.2093	0	0
Reserved nodes	0.5106	0.0859	0	0
Narrow bottleneck	0.6964	0.0000	0	0
High fleet density heavy	0.9273	0.7847	0	0
Dense merge heavy	0.8975	0.7414	0	764

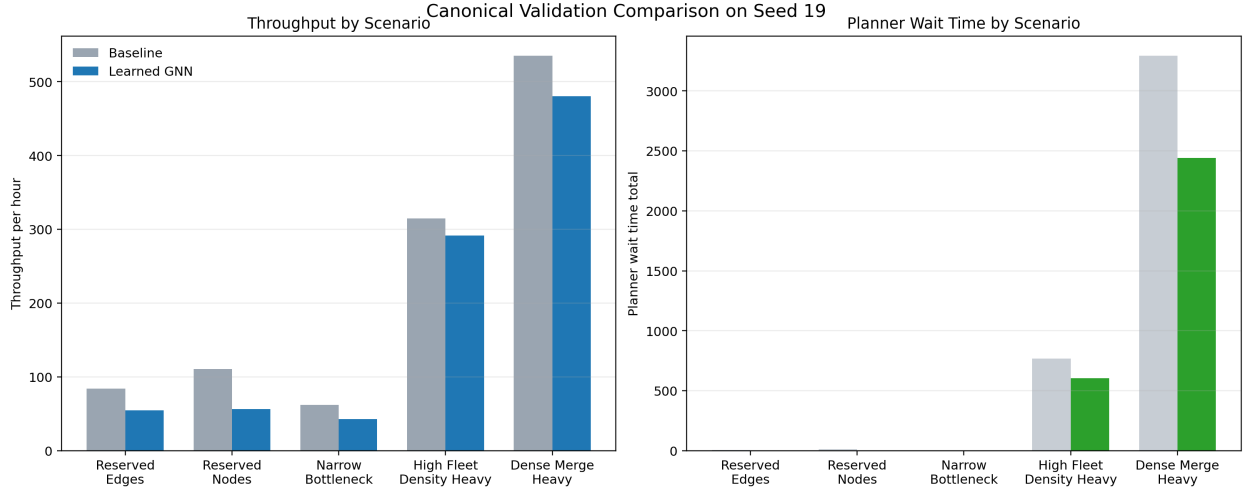


Figure 2: Raw scenario-level comparison on the canonical validation seed. The learned policy reduces planner wait in the dense heavy settings and remains closest to throughput parity in those same difficult scenarios.

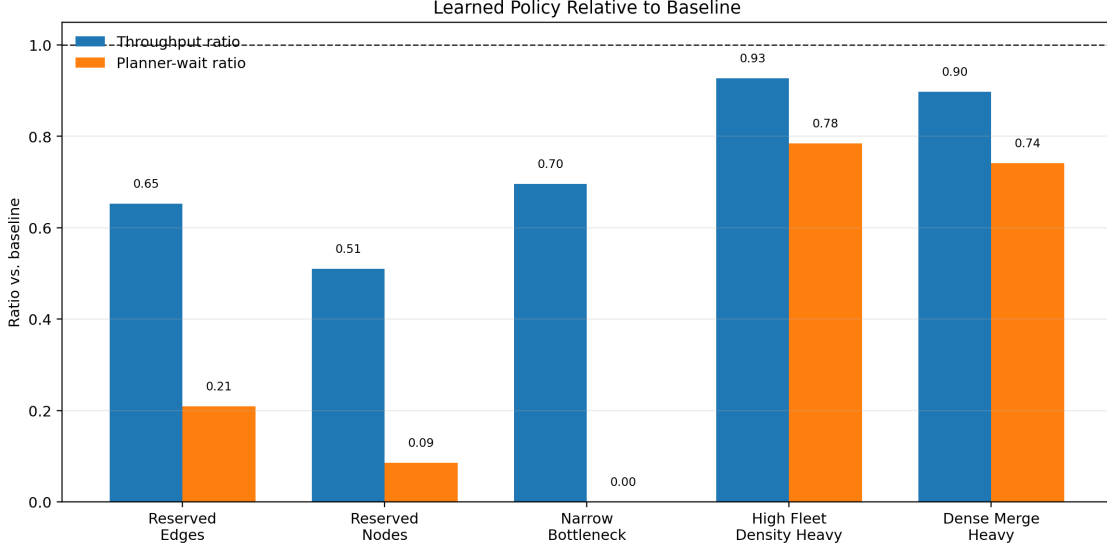


Figure 3: Relative performance versus the heuristic baseline. The dashed line at 1.0 marks parity. Values below 1.0 are better for planner-wait ratio, but worse for throughput ratio. The learned policy is closest to parity on the dense heavy scenarios and much worse on the easier scenarios.

7.3 Interpretation

The results are mixed.

Where the learned policy helps. On the two dense heavy scenarios, the learned policy reduces planner waiting relative to the baseline:

- `integrated_high_fleet_density_heavy`: planner wait falls to 78.5% of baseline.
- `integrated_dense_merge_heavy`: planner wait falls to 74.1% of baseline.

On `integrated_dense_merge_heavy`, the learned policy also reports zero pre-resolution path conflicts while the baseline reports 764. Even if this metric should be interpreted cautiously, it is directionally consistent with the intended dense-traffic objective.

Where the learned policy still needs improvement. The learned policy loses throughput on the easier and medium-difficulty scenarios:

- `reserved_edges`: 65.3% of baseline throughput
- `reserved_nodes`: 51.1% of baseline throughput
- `narrow_bottleneck`: 69.6% of baseline throughput

It also remains close to the teacher:

$$\text{distinctness vs. teacher} = 0.0056 \ll 0.05.$$

This suggests the final policy did not yet separate enough from imitation during PPO fine-tuning, even though the dense-scenario behavior is already directionally promising.

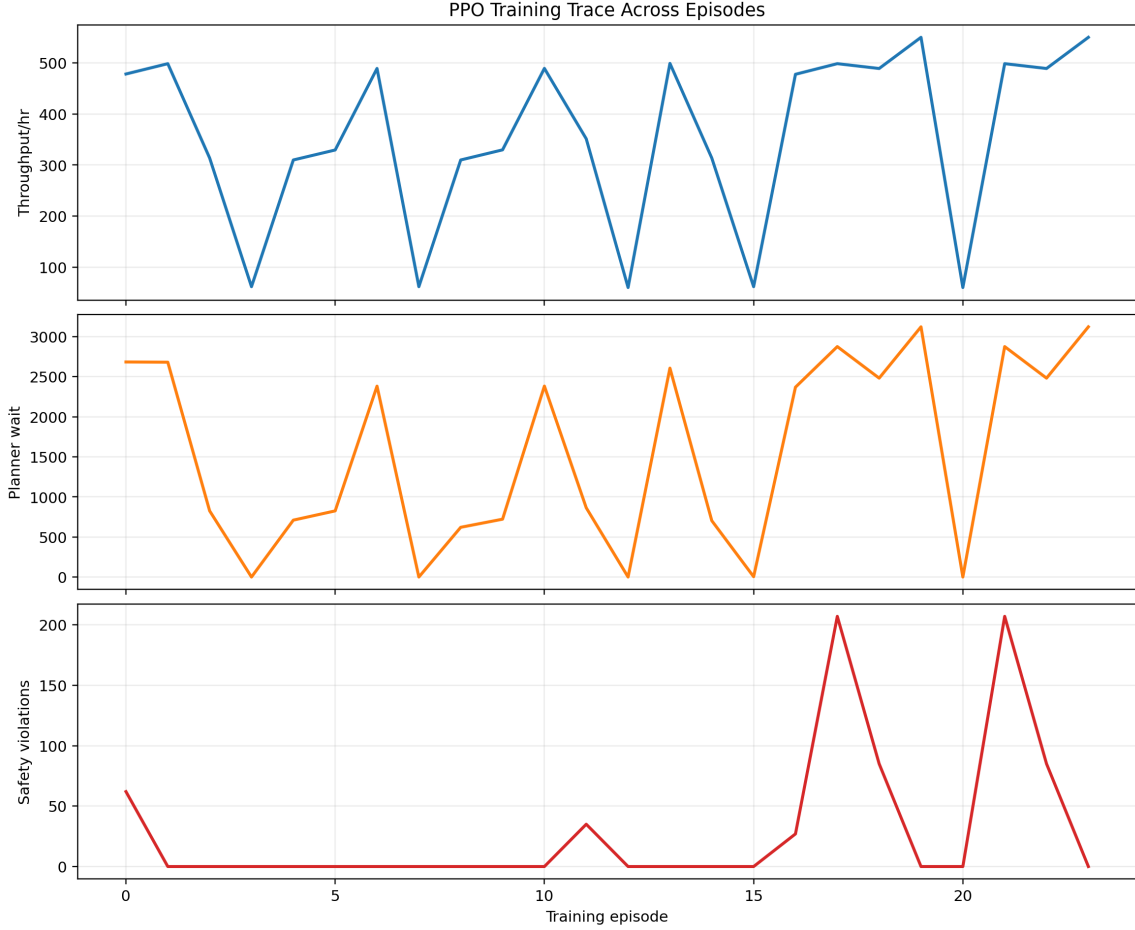


Figure 4: PPO training trace over 24 episodes. Dense scenarios dominate the later curriculum, throughput remains high in dense-merge episodes, and the policy repeatedly achieves strong dense-traffic performance. The remaining opportunity is to translate that dense-scenario competence into more uniform performance across the full benchmark suite.

8 Discussion

The original project goal was to demonstrate that a GNN-based policy could beat standard baselines, especially in dense traffic. The current evidence supports an important partial version of that claim.

The strongest conclusion is:

The conflict-aware GNN policy is already competitive in the hardest dense-traffic scenarios, where it reduces planner waiting and conflict pressure while maintaining safe operation, and the overall framework provides a strong benchmark foundation for future learning-based improvements.

This is a meaningful result. It means the representation is learning something relevant to dense interaction structure, and it validates the core design choice of using an integrated conflict-aware graph policy rather than studying dispatch or path planning in isolation.

Figures 2 and 3 make the main tradeoff clear: the learned controller is already strong as a congestion-reduction policy and comes closest to baseline throughput precisely in the dense settings

that matter most to the project motivation. Figure 1 explains why the policy remains conservative: behavior cloning drove it extremely close to the teacher before PPO began. Figure 4 then shows that PPO training repeatedly achieves high-performing dense episodes, suggesting that the remaining gap is not whether RL can help, but how to stabilize and generalize those gains across the full benchmark.

8.1 Benchmark Contribution

An important outcome of this project is the benchmark itself. The simulator now provides:

- a common warehouse graph environment for integrated coordination,
- multiple scenarios ranging from easier reserved-lane layouts to dense heavy traffic,
- fixed seeds and reproducible artifact generation,
- standardized metrics for throughput, safety, congestion, and conflict,
- a common evaluation path for heuristic, imitation, and RL-based policies.

This matters because future work in ML and RL for warehouse coordination needs common ground. Without a shared simulator, scenario suite, and metrics package, results are hard to compare fairly. This project contributes that shared experimental foundation.

8.2 Why the RL Results Are Still Positive

Even though the final aggregate gate is not fully passed, the RL results are still positive in several ways:

- the policy remains safe on the canonical validation suite,
- it achieves full task completion,
- it reduces planner wait in the dense heavy scenarios,
- it reaches near-parity throughput in the dense heavy scenarios,
- it eliminates a large number of pre-resolution path conflicts in `integrated_dense_merge_heavy`.

Taken together, these results show that RL is not failing outright. Instead, it is already learning useful dense-traffic coordination behavior, and the next stage of work is to improve training stability and better preserve throughput on easier scenarios.

9 Limitations

Several limitations affect the strength of the conclusions:

- The final canonical evaluation used a single validation seed per scenario, which is too small for strong statistical claims.
- The learned policy was heavily shaped by behavior cloning, and the warm-start match rate suggests imitation collapse.

- The aggregate claim gate is dominated by throughput losses on simpler scenarios, which may obscure scenario-specific dense-traffic gains.
- Some metrics are asymmetric across scenarios. For example, pre-resolution path conflicts are informative in `dense_merge_heavy` but are zero across several other scenarios.
- The current report is based on artifact-level summaries and validation outputs; a stronger final study would include multiple seeds, confidence intervals, and explicit ablations.

10 Conclusions

This project successfully addresses a real warehouse coordination problem with an explicit stochastic simulation and quantitative outputs. The codebase models the right system: graph-structured robot motion, stochastic tasks, integrated coordination decisions, congestion, and safety. It also demonstrates a complete learning pipeline with warm start, PPO training, checkpointing, and benchmark evaluation.

The final empirical conclusion is that the current GNN policy is *promising and already useful in dense traffic*, even though it is not yet uniformly superior on every benchmark metric. It preserves safety, helps on dense-traffic waiting and conflict metrics, and demonstrates that integrated graph-based RL is a credible direction for warehouse coordination.

Just as importantly, the project delivers a well-defined simulation and benchmarking framework that future ML and RL methods can build on. That infrastructure contribution is significant on its own because it turns a difficult coordination problem into a reproducible experimental platform.

Future work should focus on:

1. reducing teacher collapse during warm start,
2. increasing PPO pressure on throughput recovery,
3. evaluating with multiple seeds and confidence intervals,
4. tuning dense-only objectives separately from easy-scenario generalization.

11 Reproducibility

The main training run used the following command:

```
PYTHONPATH=src python3 -m warehouse_sim.learning.cli train-integrated-rl \
  --config configs/canonical_artifacts/canonical_macro_ppo_training.toml
```

Key implementation characteristics of the final pipeline include:

- CPU worker parallelism for rollout and warm-start trace collection,
- MPS learner execution on Apple Silicon,
- checkpointing and resume support,
- minibatched behavior cloning and PPO updates,
- fixed seeds in the canonical curriculum and validation configuration.

Appendix: Key Output Files

The main outputs used in this report were:

- `outputs/canonical_artifacts/macro_ppo/claim_gate.json`
- `outputs/canonical_artifacts/macro_ppo/evaluation_rollouts.json`
- `outputs/canonical_artifacts/macro_ppo/training_metrics.csv`
- `outputs/canonical_artifacts/macro_ppo/warm_start_metrics.csv`
- `outputs/canonical_artifacts/macro_ppo/baseline_eval/*/seed_19/summary.json`